# Rajarata University of Sri Lanka

## Faculty of Applied Sciences

## Department of Computing

## ICT3212 – Artificial Intelligence

### Implementation 2 – Summary Report

# NueroNet

| Name | Registration Number | Index No |
|---|---|---|
| M. N. F Laiza | ICT/2022/117 | 5719 |
| M. A. F Nuha | ICT/2022/106 | 5709 |
| M. F. R Azara | ICT/2022/009 | 5617 |
| M. T. M Aathif | ICT/2022/129 | 5730 |
| G. C. P. A Hemasiri | ICT/2022/067 | 5671 |

**Table of Content**

**Smart Waste Classification Using Convolutional Neural Networks**

## 1. Introduction

Waste management is an important environmental challenge. Proper separation of waste into categories such as Non-Recyclable and recyclable materials helps improve recycling efficiency and reduce landfill waste. However, manual waste sorting is time-consuming and prone to human error.

With the advancement of artificial intelligence, computer vision techniques can be used to automatically classify waste images. Convolutional Neural Networks (CNNs) have proven highly effective in image classification tasks because they can automatically learn visual features from images.

This project aims to develop a deep learning model that can classify waste images into two categories:

- Non-Recyclable waste (O)
- Recyclable Waste (R)

The project was implemented using Python and TensorFlow/Keras. Two main implementations were developed:

- **Implementation 1 – Baseline CNN Model**
- **Implementation 2 – Improved CNN Model**

The objective of the project is to analyze different model architectures, evaluate their performance, and identify improvements that lead to better classification results.

## 2. Dataset Description

The dataset used in this project contains images of waste materials categorized into two classes:

- Non-Recyclable waste(O)
- Recyclable waste(R)

The dataset was divided into three subsets:

| **Dataset** | **Number of Images** |
|---|---|
| Training | 15,400 |
| Validation | 3,300 |
| Test | 3,300 |

**Class Distribution**

| **Class** | **Training** | **Validation** | **Test** |
|---|---|---|---|
| Organic (O) | 7700 | 1650 | 1650 |
| Recyclable (R) | 7700 | 1650 | 1650 |

The dataset is **perfectly balanced**, meaning each class has the same number of images. This prevents bias during model training and makes accuracy a reliable evaluation metric.

## 3. Data Preprocessing

Before training the model, several preprocessing steps were applied.

### Image Resizing

All images were resized to:

224 × 224 pixels

This ensures consistent input size for the CNN model.

### Normalization

Pixel values originally range from:

0 – 255

They were normalized using:

rescale = 1/255

This converts pixel values to a range between **0 and 1**, which helps improve training stability and convergence speed.

### Batch Processing

Batch size used:64

Batch processing improves memory efficiency and speeds up training.

### Preprocessing Configuration Used in Both Implementations

| **Parameter** | **Value** |
|---|---|
| Image Size | 224 × 224 |
| Normalization | Rescale (1/255) |
| Batch Size | 64 |
| Training Images | 15,400 |
| Validation Images | 3,300 |
| Test Images | 3,300 |

The preprocessing configuration remained the same in both Implementation 1 and Implementation 2 to maintain consistency during experimentation.

## 4. Model Architectures and Experiments

Two different CNN architectures were implemented and evaluated during the project.

### 4.1 Implementation 1 – Baseline CNN Model

Implementation 1 was designed as a **simple baseline model** to evaluate the initial performance of CNN for waste classification.

**Architecture**

The model contains three convolutional blocks followed by a fully connected layer.

Layers used:

- Conv2D (4 filters)
- MaxPooling
- Conv2D (8 filters)
- MaxPooling
- Conv2D (16 filters)
- MaxPooling
- Flatten
- Dense (32 neurons)
- Output layer (Sigmoid activation)

**Model Summary**

| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d (Conv2D) | (None, 222, 222, 4) | 112 |
| max_pooling2d (MaxPooling2D) | (None, 111, 111, 4) | 0 |
| conv2d_1 (Conv2D) | (None, 109, 109, 8) | 296 |
| max_pooling2d_1 (MaxPooling2D) | (None, 54, 54, 8) | 0 |
| conv2d_2 (Conv2D) | (None, 52, 52, 16) | 1,168 |
| max_pooling2d_2 (MaxPooling2D) | (None, 26, 26, 16) | 0 |
| flatten (Flatten) | (None, 10816) | 0 |
| dense (Dense) | (None, 32) | 346,144 |
| dense_1 (Dense) | (None, 1) | 33 |

**Total params:** 347,753 (1.33 MB)
**Trainable params:** 347,753 (1.33 MB)
**Non-trainable params:** 0 (0.00 B)

Total parameters:

347,753

This architecture is relatively small and computationally efficient but may have limited feature extraction capability.

### 4.2 Implementation 2 – Improved CNN Model

Implementation 2 introduces several improvements to reduce overfitting and improve model performance.

**Improvements Introduced**

1. **Deeper CNN architecture**
2. **L2 Regularization**
3. **Dropout Regularization**
4. **Early Stopping**
5. **Learning Rate Tuning**

**Architecture**

Layers used:

- Conv2D (32 filters)
- MaxPooling
- Conv2D (64 filters)
- MaxPooling
- Conv2D (128 filters)
- MaxPooling
- Conv2D (128 filters)
- MaxPooling
- Flatten
- Dense (128 neurons)
- Dropout (0.5)
- Output layer (Sigmoid)

**Model Summery**

| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d_4 (Conv2D) | (None, 222, 222, 32) | 896 |
| max_pooling2d_4 (MaxPooling2D) | (None, 111, 111, 32) | 0 |
| conv2d_5 (Conv2D) | (None, 109, 109, 64) | 18,496 |
| max_pooling2d_5 (MaxPooling2D) | (None, 54, 54, 64) | 0 |
| conv2d_6 (Conv2D) | (None, 52, 52, 128) | 73,856 |
| max_pooling2d_6 (MaxPooling2D) | (None, 26, 26, 128) | 0 |
| conv2d_7 (Conv2D) | (None, 24, 24, 128) | 147,584 |
| max_pooling2d_7 (MaxPooling2D) | (None, 12, 12, 128) | 0 |
| flatten_1 (Flatten) | (None, 18432) | 0 |
| dense_2 (Dense) | (None, 128) | 2,359,424 |
| dropout_1 (Dropout) | (None, 128) | 0 |
| dense_3 (Dense) | (None, 1) | 129 |

**Total params:** 2,600,385 (9.92 MB)

**Trainable params:** 2,600,385 (9.92 MB)

**Non-trainable params:** 0 (0.00 B)

Total parameters:

2,600,385

The deeper architecture enables the model to capture more complex visual features.**5. Training Process**

### 5.1 Implementation 1 – Baseline CNN Model

**Hyperparameters Used**

| **Parameter** | **Value** |
|---|---|
| Image Size | 224 × 224 |
| Batch Size | 64 |
| Optimizer | Adam |
| Learning Rate | Default (0.001) |

| | |
|---|---|
| Loss Function | Binary Crossentropy |
| Epochs | 20 |

### 5.1 Implementation 2 – Improved CNN Model

**Hyperparameters Used**

| **Parameter** | **Value** |
|---|---|
| Image Size | 224 × 224 |
| Batch Size | 64 |
| Optimizer | Adam |
| Learning Rate | 0.0005 |
| Loss Function | Binary Crossentropy |
| Epochs | 30 |
| Early Stopping Patience | 5 |

**Early Stopping**

Early stopping monitors validation loss during training.

If validation loss does not improve for 5 consecutive epochs, training stops automatically. This prevents unnecessary training and reduces overfitting.

## 6. Experimental Results

### Implementation 1 Results

Training accuracy gradually increased during training and reached approximately:

99%

However, validation accuracy remained around:

87%

Final test performance:

| Metric | Value |
|---|---|
| Test Accuracy | 82.57% |
| Test Loss | 1.0435 |

The gap between training accuracy and validation accuracy indicates that the model suffered from **overfitting**.

### Implementation 2 Results

The optimized model achieved better generalization.

Final test results:

| Metric | Value |
|---|---|
| Test Accuracy | 89.18% |
| Test Loss | 0.3438 |

## 7. Comparison of Experiments

| Feature | Implementation 1 | Implementation 2 |
|---|---|---|
| CNN Layers | 3 | 4 |

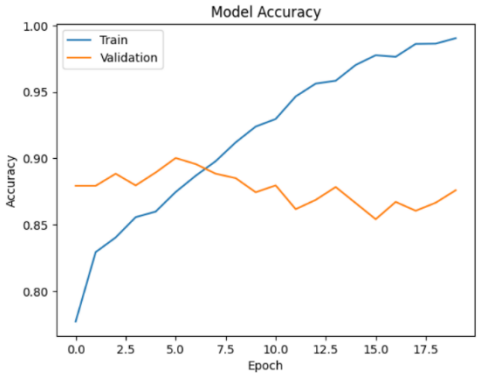
| | | |
|---|---|---|
| Filters | 4–16 | 32–128 |
| Regularization | No | L2 + Dropout |
| Early Stopping | No | Yes |
| **Test Accuracy** | **82.57%** | **89.18%** |
| **F1 Score** | **82.9%** | **89.2%** |

Implementation 2 improved accuracy by approximately **6.6%**.

## 8. Overfitting Analysis

Overfitting occurs when the model memorizes the training data instead of learning general patterns.

### Implementation 1





The training accuracy steadily increases and reaches nearly 99%, while the validation accuracy remains around 86–90% and begins to fluctuate after several epochs. At the same time, the training loss continuously decreases, but the validation loss starts increasing after around epoch 6. This indicates that the model begins to overfit the training data. The model learns the training patterns very well but does not generalize effectively to unseen data

Training Accuracy:

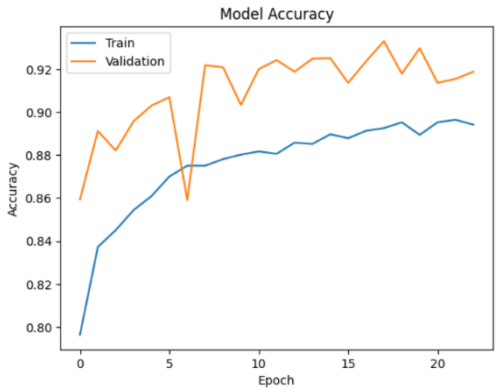
99%

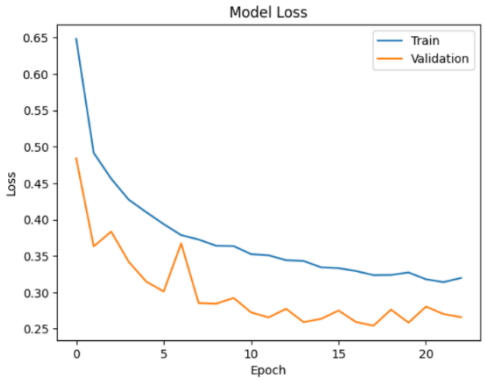
Validation Accuracy:

87%

The large gap indicates strong overfitting.

**Implementation 2**




The training accuracy gradually increases and stabilizes around 89–90%, while the validation accuracy remains slightly higher and stays around 91–93%. Both training and validation loss decrease steadily during the training process. Unlike the previous model, the validation loss does not increase, indicating that overfitting is reduced. The small gap between training and validation performance suggests that the model generalizes well to unseen data. This shows that the improvements applied to the model helped achieve more stable and reliable performance.

Training Accuracy:

~89%

Validation Accuracy:

~93%

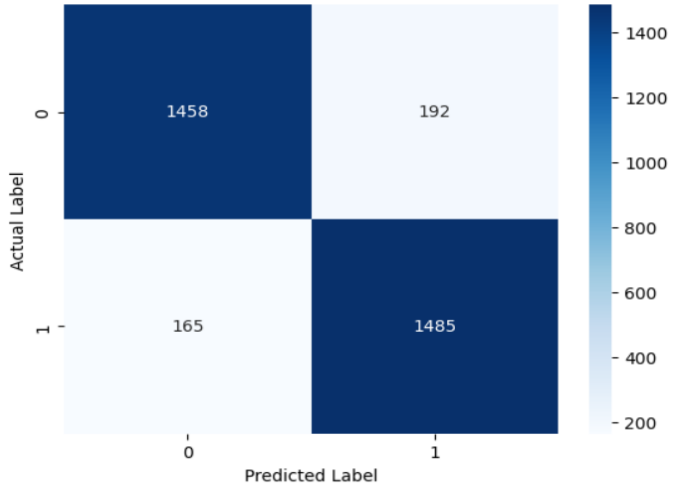
This shows better generalization. Overfitting was reduced by:

- L2 regularization
- Dropout
- Early stopping
- Hyper parameter tuning

**9. Final Model and Results**

The final model selected for the project is **Implementation 2**.

**Confusion matrix:**




**Final performance metrics:**

| **Metric** | **Value** |
|---|---|
| Test Accuracy | 89.18% |
| Precision | 0.885 |
| Recall | 0.90 |
| F1 Score | 0.892 |

The model demonstrates strong performance for binary waste classification.

**Final Model Prediction**


## 10. Discussion

### What Worked Well

- Convolutional Neural Networks successfully extracted visual features from waste images.
- Regularization techniques improved model generalization.
- Using a suitable learning rate helped the model learn faster and reach better results during training.
- Early stopping prevented unnecessary training and reduced overfitting.

### Challenges Faced

- Initial model suffered from overfitting.
- Training deep learning models required significant computational resources, especially when processing large image datasets(training time).
- Image variability such as different lighting conditions, angles, and backgrounds made classification more difficult.
- Hyperparameter tuning increased training time, since multiple experiments were required to achieve better performance

### Lessons Learned

This project demonstrated the importance of:

- Model architecture design
- Regularization techniques
- Proper evaluation metrics
- Hyperparameter tuning
- Early stopping techniques

This project showed that high training accuracy does not always indicate a better model, as a model may simply memorize the training data rather than learning meaningful patterns that generalize well to new data. It also demonstrated that regularization techniques are essential for preventing overfitting, since they help the model avoid relying too heavily on specific training examples and improve its ability to perform well on unseen data. In addition, the project highlighted that validation performance is a more reliable indicator of model quality because it measures how effectively the model can make predictions on data that was not used during training.

**Git-Hub Link**

https://github.com/FathimaLaiza/NeuroNet

## 11. Conclusion

This project successfully developed a deep learning model for classifying waste images into non-recyclable and recyclable categories.

Two CNN models were implemented and evaluated. The baseline model achieved a test accuracy of 82.57%, while the improved model achieved 89.18% accuracy.

The optimized model used L2 regularization, dropout, and early stopping to improve generalization and reduce overfitting.

Overall, the results demonstrate that CNN-based approaches are effective for waste classification tasks.

## 12. Future Work

Although the model achieved good performance, several improvements can be explored in future work:

- Using pretrained models such as ResNet or MobileNet
- Applying data augmentation to increase dataset diversity
- Increasing dataset size
- Developing a real-time waste classification system using a camera
- Deploying the model as a mobile or web application

These improvements could further enhance the accuracy and practical usability of the system.